

■ AI-Powered Incident Root Cause Analyzer

Hackathon Problem Statement

Overview

Modern enterprise applications generate massive volumes of logs, alerts, metrics, and operational events across distributed systems. During production incidents, identifying the actual root cause becomes time-consuming and impacts business continuity.

Build an intelligent platform that helps engineering and operations teams rapidly identify, analyze, and summarize production incidents using **agentic AI workflows powered by LangChain, LangGraph, and modern GenAI orchestration frameworks**.

Participants are expected to leverage **agent-based architectures** for reasoning, tool use, and multi-step root cause analysis. Solutions must demonstrate meaningful use of GenAI orchestration frameworks — not just raw LLM API calls.

Key Responsibilities

- **Ingest & Correlate** — Parse and correlate logs, alerts, and metrics from distributed systems using structured pipelines (e.g., vector stores, embeddings-based retrieval)
- **Agentic Root Cause Analysis** — Build a **LangGraph-based multi-agent workflow** (or equivalent) that orchestrates specialized agents for log analysis, anomaly detection, metric correlation, and root cause reasoning
- **RAG-Powered Context Retrieval** — Implement **Retrieval-Augmented Generation (RAG)** using LangChain to ground LLM responses in actual log/metric data, runbooks, or knowledge bases
- **Incident Summarization** — Generate concise, human-readable incident summaries and recommended actions using LLM chains with structured output parsing
- **Tool-Use & Function Calling** — Agents must use **LangChain Tools** (e.g., custom tools for log search, metric query, alert lookup) to interact with data sources programmatically
- **Scalable Backend** — Design the backend using **Java, Spring Boot, and Microservices**; the GenAI orchestration layer can be in **Python (LangChain/LangGraph)** invoked via REST or gRPC from the Spring Boot services

Mandatory Framework Requirements

Layer	Expected Frameworks
Agent Orchestration	LangGraph (stateful multi-agent graphs with conditional edges, cycles, and human-in-the-loop checkpoints)
LLM Chains & RAG	LangChain (chains, retrievers, document loaders, output parsers, tool agents)
Vector Store	ChromaDB / FAISS / Pinecone / pgvector (any)

Layer	Expected Frameworks
LLM Provider	OpenAI / Azure OpenAI / Google Gemini / Ollama (any)
Backend Services	Java + Spring Boot + Microservices
API Layer	REST APIs with a working demo

Bonus Points

Criteria	Points
Spring AI Integration — Using Spring AI for LLM orchestration, prompt templating, function calling, or RAG directly within the Java/Spring Boot layer (reducing Python dependency)	■ Additional Points
Human-in-the-loop checkpoint in LangGraph (e.g., analyst approval before finalizing root cause)	■ Bonus
Multi-agent collaboration (e.g., separate Log Analyzer, Metric Correlator, and Summary Writer agents with defined handoffs)	■ Bonus
Streaming responses for real-time incident analysis updates	■ Bonus
Evaluation harness — using LangSmith or custom evals to measure agent accuracy	■ Bonus

Scope Boundary

- Participants may use **synthetic or pre-collected** log/metric datasets (sample datasets will be provided)
- A full observability pipeline (Prometheus, Grafana, ELK) is **not required** — focus on the **intelligence layer**: ingestion, agentic analysis, and explanation
- No production-grade UI expected — a **REST API with a working demo** (Postman/Swagger/simple frontend) is sufficient
- The LangGraph agent graph **must be demonstrable** — participants should be able to walk through the agent flow, state transitions, and tool invocations during the demo

Good luck to all participants! Build smart, build agentic. ■